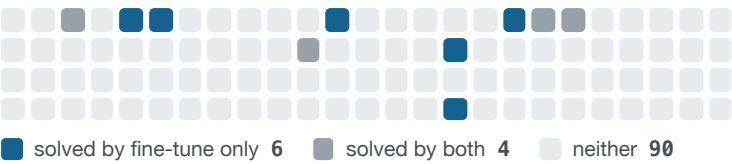


15,529 public trajectories and 43 GPU-hours on a laptop made a 9B model solve 2.5x more CTF tasks

Every task the base model solved, the fine-tune also solved, plus six more, and it got there in 60% of the wall time. No proprietary data, no cluster, no annotation budget.

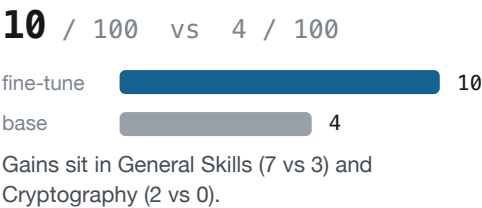
INTERCODE-CTF · ONE SQUARE PER TASK · 100 TASKS · TEMPERATURE 0



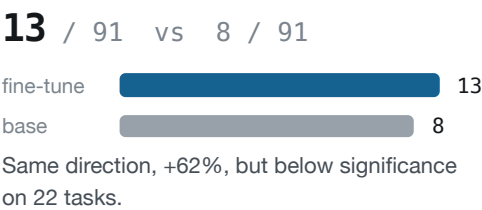
base-only solves: 0

A **strict subset**: the gain is not a trade. McNemar on the paired result, $\chi^2 = 4.17$, $p < 0.05$.

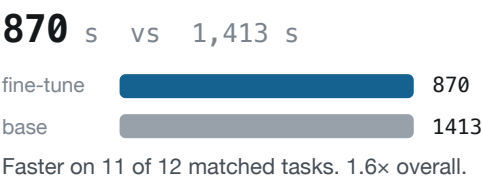
TASKS SOLVED



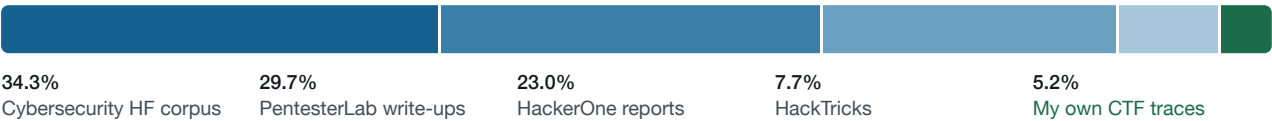
CYBENCH SUBTASKS



WALL CLOCK PER TASK · LOWER IS BETTER



WHAT IT LEARNED FROM · 15,529 ROWS AFTER MULTI-TURN UNROLL · 90 GB TRIMMED TO 1 GB



~90% is prose turned into commands; only 5.2% is real execution. That ratio explains the shape of every result above.

TRAINING	THE CONSTRAINT	SET OUT TO DO → WHAT HAPPENED
LoRA r32 α64 · 12 mods	Intended base was Qwen3.8-27B . 36 GB cannot hold a 27B base plus LoRA state plus activations, so the study ran on Qwythos-9B .	Improve autonomous CTF solving 10/100 vs 4/100 · $p<0.05$
Trainable 86.5 M · 5.8%		Leave general knowledge intact no regression · $p=0.21$
Steps 1,941 · 1 epoch	A 27B control later took 4 flags to the 9B's 1.	Improve full-chain network pentest no measurable change
Wall clock 43 h 28 m		Improve web exploitation zero web solves, either model
Peak RAM 12.1 / 36 GB		Produce a servable artifact fused 4-bit weights

Part I · Executive summary

1 · The question

Can a single person, on one laptop, using only publicly available data, measurably improve a local AI model at autonomous offensive-security work?

No proprietary data. No cluster. No labelled human feedback. One 36 GB Mac, public write-ups and disclosed vulnerability reports, and twenty days.

Answer: yes, measurably, and by more than one metric.

2 · The headline

	trained	base	verdict
InterCode-CTF (100 tasks)	10 solved	4 solved	2.5× · $p < 0.05$
CyBench (91 subtasks)	13	8	+62% · not significant
Wall-clock per task	870 s	1,413 s	1.6× faster

Three independent measurements, all pointing the same way. One of them is statistically significant; one is directional; one is large and consistent.

The cleanest result: on 100 CTF tasks at temperature 0, the fine-tuned model solved every task the base model solved, plus six more. Zero regressions. A strict superset.

3 · What it cost

Elapsed, download → final report	~20 days
Raw corpus downloaded	90 GB
Corpus after cleaning	1 GB (−99%)
Final training set	15,529 rows · 19.5 M tokens
GPU-hours	43.5 h — one epoch, one laptop
Trainable parameters	86.5 M of 9 B (5.8%)
Peak memory	12.1 GB of 36 GB
Marginal cost	\$0 — electricity

4 · The constraint that shaped everything

The intent was to train **Qwen3.8-27B**. It did not fit: 36 GB of unified memory cannot hold a 27B base, LoRA optimiser state and activations at a useful sequence length.

I moved to **Qwythos-9B-v2**. That decision propagated. A 9B model scores near zero on hard benchmarks, so measuring *improvement* on them is measuring noise. Most of the methodological work in this project exists because the model is small enough that the effect size is comparable to the noise floor.

A 27B control run later confirmed the cost: on network-penetration tasks a 27B model captured 4 flags to the 9B's 1, including three tasks both 9Bs failed after exhausting their iteration budget.

5 · What the model learned from

Nine public sources, unified into one trajectory format (observation → reasoning → action):

By upstream origin, one downloaded public corpus plus my own execution traces:

upstream source	rows	share
PentesterLab write-ups	4,613	29.7%
HackerOne disclosed reports	3,579	23.0%
Cybersecurity HF corpus — 817 skill dirs, study sets, markdown	5,331	34.3%
HackTricks	1,193	7.7%
My own CTF / lab traces	813	5.2%

Grouped differently: **94.8% came from one public corpus I downloaded and trimmed; 5.2% is my own execution against live machines.**

Operator principles are in every row. The system prompt encodes two rules authored for this project, and they apply across all 15,529 rows:

"Work the target methodically: enumerate the attack surface, run one concrete step at a time, read the actual tool output before deciding the next move, and keep an explicit record of what you have and have not tested. Never claim something was tried, ruled out, or exhausted unless you actually did it."

This is an anti-falsification and persistence constraint imposed at the data level rather than learned from any source corpus. Set against §5: the model was *instructed* not to claim untested conclusions, yet still failed to treat 14 identical null observations as evidence. The instruction addresses **reporting** honesty; it does not by itself produce hypothesis revision, which is the capability §5 finds missing.

Mean 5.9 turns per trajectory, up to 24. **~90% is prose-derived**, reports, write-ups and technique pages turned into commands. Only 5.2% is real execution against live machines.

That ratio explains the shape of every result below, and §II-6 argues it is the main thing to change next.

6 · What I set out to do vs what happened

goal	outcome
Improve autonomous CTF solving	✅ 10/100 vs 4/100, $p < 0.05$, strict superset
Don't damage general knowledge	✅ no measurable regression (1,105-question exam, $p = 0.21$)
Improve full-chain network pentest	❌ no measurable change
Improve web exploitation	❌ zero web solves, either model, any benchmark
Produce a servable artifact	✅ fused 4-bit weights, served locally

7 · What I would do differently with the *same* data

No new corpus required, six changes, roughly in order of expected value:

1. Add failure. The corpus is built from write-ups, which record only what *worked*. The model correspondingly never learns to abandon a failing hypothesis (§II-7 documents it doing the same wrong thing 31 times).
2. Rebalance toward execution. 5.2% gold lab → 20%+. The measured gains are in *operating a shell*, which is exactly what gold teaches.
3. Include the web data I already built. 1,501 execution-shaped web trajectories exist but were finished *after* this checkpoint, so they were never trained on.
4. Fix the no-prose-leak gate. It taught the model to emit an empty message body, which required a workaround on every benchmark and still breaks one serving stack.
5. Deduplicate the validation set. 5 of 150 held-out rows are byte-identical to training rows.
6. **Evaluate with 3–5 seeds per model.** Repeat runs on a 22-task benchmark span 3 to 11 of 91, so a single run per model cannot separate a real effect of this size from sampling noise.

8 · Bottom line

A one-person, one-laptop, public-data fine-tune produced a statistically significant improvement in autonomous CTF capability, with no knowledge regression, in 43 GPU-hours.

It also produced an agent that is **1.6× more economical**, reaching an equal-or-better score in 60% of the wall time, which for a deployed agent may matter more than the solve count.

The gains are **narrow**: shell operation improved; vulnerability classes did not. The path to widening them is a data-composition problem, not a compute problem, which is the most actionable possible conclusion for a project of this size.

Part II · Research paper

Abstract

I fine-tuned a 9B-parameter local language model (Qwythos-9B-v2, `qwen3_5` Gated-DeltaNet) for one epoch on 15,529 offensive-security trajectories assembled entirely from public sources, using LoRA ($r=32$, $\alpha=64$) across 12 target modules including all five Gated-DeltaNet leaves. On InterCode-CTF (100 picoCTF-derived tasks, 15 turns, temperature 0) the fine-tuned model solved **10/100** against 4/100 for the identical base weights. The comparison is paired over all 100 tasks: 6 trained-only solves, 0 base-only solves, McNemar $\chi^2=4.17$, $p<0.05$, with base's solves forming a strict subset of the fine-tuned model's. On CyBench under fully matched conditions the fine-tune scored 13/91 vs 8/91, the same direction, not significant (task-level sign test $p=0.45$). A secondary and much larger effect: the fine-tuned model completed the same work in 870 s/task against 1,413 s/task, 1.6× faster, on 11 of 12 tasks.

Gains are confined to **General Skills (7/33 vs 3/33)** and Cryptography (2/19 vs 0/19). Reverse Engineering (0/27), Binary Exploitation (0/4) and Web Exploitation (0/2) were zero for both models. I conclude that this recipe improves *shell-driven task execution* and not any vulnerability class, and I trace that to a corpus in which ~90% of rows are prose-derived and only 5.2% are real execution traces.

1 · Motivation and constraints

The question is economic as much as scientific: **what does a fine-tune buy when the inputs are free?** No proprietary data, no annotation budget, no cluster.

The intended base was **Qwen3.8-27B**. On 36 GB of unified memory, a 27B base plus LoRA optimiser state plus activations at a 4,096-token sequence length does not fit. I substituted Qwythos-9B-v2 (4-bit MLX, group size 64).

This substitution is the project's dominant confound. Small models score near the floor on security benchmarks, so the measurable signal is small relative to benchmark noise, and much of \$5 exists to handle that. A later 27B control on network-penetration tasks captured **4 flags to the 9B's 1**, including three tasks both 9B variants failed after exhausting 20 iterations, quantifying what the constraint cost.

2 · Corpus construction

90 GB → 1 GB. The raw public material (disclosed vulnerability reports, PentesterLab write-ups, technique wikis, study sets) was filtered down to executable, trajectory-shaped content: deduplicated, stripped of prose that contained no action, normalised to one schema, and size-capped per row.

Upstream provenance is narrow: a single downloaded public corpus supplies **94.8%** of rows (PentesterLab write-ups, HackerOne disclosed reports, HackTricks command pages, 817 skill directories, study sets and top-level markdown), and my own lab harness supplies the remaining 5.2% as real execution traces.

Every row is `{prompt, completion, tools, _meta}`, where `completion` carries a `reasoning_content` (the thinking trace) and either a message or a `run_command` tool call. Composition of the final 15,529 rows is given in §I-5. Shapes: A 13,450 / B 1,162 / C 917.

2.1 Five examples per source (verbatim, truncated)

My own CTF traces, real execution against live lab machines via my harness:

```

OBS faketime=2026-09-17 12:58:20 HTTP/gitea.darkzero.ext:3000 svc-gitea CN=ServiceHandler...
THINK DC clock is +7h vs labbox. Use faketime '+7h' to align and request the svc-gitea
      Kerberoast TGS (impacket-GetUserSPNs) over the tunnel. Capture krb5tgs hash.
ACT proxychains4 -q faketime -f '+7h' impacket-GetUserSPNs darkzero.ext/josh:'Rangers1' ...

OBS Username or password is incorrect. --- <title>Sign In - Gitea</title>
THINK As josh (SSH), search his home for Gitea credentials/tokens (.git-credentials, .netrc,
      shell history, cloned repos with stored tokens) to obtain Gitea access.
ACT sshpass -p 'Rangers1' ssh -o StrictHostKeyChecking=no josh@10.129.132.38 'cd ~; ls -la ...'

OBS [+] darkzero.ext\celia:babygurl13 (Pwn3d!) == DCSync krbtgt == krbtgt:502:aad3b435b5...
THINK Forge golden ticket for celia (real RID 1109 for Server-2025 PAC RID validation) with
      SID history extra-sid = InfrastructureAdministrators.
ACT impacket-ticketer -domain "$D" -domain-sid S-1-5-21-2850783758-12...

OBS uid=33(www-data) gid=33(www-data) groups=33(www-data) 124a042cc76c MARKER123
THINK Backend readfile SSRF can reach the front (172.18.0.2). Read the front's webroot for a
      flag. Double-encode the scheme.
ACT python3 - <<'PY' ... enc=url.replace('/:/',':%252f%252f') ...

OBS ==REG== ls: cannot open directory '/opt/gitea-runner': Permission denied
THINK Check if Gitea self-registration is enabled and whether the repo is publicly browsable.
ACT bash /tmp/rce.sh 'curl -s http://172.16.20.2:3000/user/sign_up | grep -iE "Register..."'

```

HackerOne reports, disclosed vulnerability reports rendered as operator steps:

OBS Red-teamer testing the Node.js HTTP stack behind a frontend proxy, probing whether a lone CR (without LF) can smuggle a request by exploiting llhttp's loose header parsing.

THINK Stand up an observable backend that echoes parsed headers and body so smuggled requests can be captured on stdout.

ACT node server.js

OBS req 1 -> 200 ... req 60 -> 200 # no 429 / no throttle window observed

THINK The manual loop is slow; pivot to a concurrent burst to demonstrate scale.

ACT seq 1 200 | xargs -P 20 -I{} sh -c 'curl -sS -o /dev/null -w "%{http_code}\n" -X P...'

OBS Red-teamer engaged against Cisco ASA VPN gateways to confirm whether +CSCOE+/session_password.html Local File Disclosure is present.

THINK Confirm the vulnerable endpoint exists (presence = unpatched) before touching production.

ACT curl -s -o /dev/null -w '%{http_code}\n' https://.../+CSCOE+/session_password.html

OBS login_resp.json "MMAUTHTOKEN":"00d7f8a2..." me.json { "data": { "id": "m9t3w4r5..."

THINK Before generating a 50MB payload, confirm the Playbooks plugin is live and accepts JSON POSTs from the harvested token.

ACT TOKEN=00d7f8a2...; CSRF=m9t3w4r5...

OBS HTTP/1.1 500 Internal Server Error select AAA,BBB from EEE where location="abcd1337""

THINK Error-based SQLi confirmed by the unbalanced quote; enumerate the column count next.

ACT curl -s "...?location=abcd1337' order by 1-- --"

HackTricks, technique pages turned into attempt-and-adapt sequences:

OBS ServiceInfo: com.evil.rat/.EvilService exported=true, permission=BIND_ACCESSIBILITY_SE...

THINK Attempt to enable the service programmatically, but the system blocks silent grant - BIND_ACCESSIBILITY_SERVICE requires user interaction.

ACT adb shell am start -a android.accessibilityservice.AccessibilityService -s com.evil.rat/...

OBS manifest.json (debugger + webRequest + cookies permissions) { "manifest_version": 3, ...

THINK Force-close the live Chrome session so I can relaunch with tampered flags and the pre-loaded extension.

ACT osascript -e 'tell application "Google Chrome" to quit'

OBS ysoserial .NET v1.0 [+] Generator: TypeConfuseDelegate [+] Legacy payload generated

THINK Legacy worked but ENCRYPTED=1 is set; on .NET 4.5 the app may need the full TextFormattingRunProperties payload with both keys. Pivot to the modern gadget.

ACT ysoserial.exe -p ViewState -g TextFormattingRunProperties -c "cmd /c whoami" ...

OBS <no output; url reassigned>

THINK Retry the retrieval now that the alternate source path is in place.

ACT urllib.request.urlretrieve(url, file_path)

OBS (kerberos pre-auth response) ...

THINK Enumerate accounts with pre-auth disabled before spraying, to avoid lockout.

ACT impacket-GetNPUsers ...

Cybersecurity corpus skills, multi-tool operations synthesised from its 817 skill directories (cloud, forensics, malware, RE):

```
OBS  Running module sts__assume_role... [+] AssumeRole SUCCESSFUL [+] Temporary credentials acquired
THINK With assumed-role credentials active, re-scan the resource inventory to capture the
      complete loot set (Lambda, Secrets Manager, SSM).
ACT   Pacu > run lambda__enum

OBS  HTTP/1.1 200 OK Server: cloudflare X-Cache: HIT from cache2-eu CF-Cache-Status: HIT
THINK CF-Cache-Status is HIT. Fetch the body unauthenticated to prove the victim's dashboard
      data is served without a session cookie – deception confirmed.
ACT   curl -s "http://billing.acmecloud.io/dashboard/billing/anything.css" | head -c 400

OBS  A scheduled-task dropper 'svchost_update.exe' was collected from a phishing victim.
THINK Confirm it is a managed .NET assembly and not a native PE before opening in dnSpy.
ACT   file C:\Analysis\suspect.exe

OBS  total 1.8M -rw-r--r-- 218K 2b7c9f1a3e5d...sample 512K a3f91c2e4b7d...sample
THINK Verify the pipeline captured all samples with correct SHA-256 names.
ACT   ls -lh /opt/malware_quarantine/ && sha256sum /opt/malware_quarantine/*.sample

OBS  INFO [PlasoStorageWriter] Storage writer started. INFO [Process] Processing data from ...
THINK Import the Plaso storage file into a new Timesketch sketch; the import hangs because the
      Celery worker is still initialising OpenSearch.
ACT   timesketch_importer -s "Case-2025-001" -t "Endpoint-WKS01" full_analysis.plaso
```

PentesterLab write-ups, command blocks and skill chains follow the same schema, derived respectively from PentesterLab write-up narratives, fenced command blocks in public write-ups, and scripted skill chains. (stage6_web_exec , 1,501 discovery-first web trajectories, was completed *after* this checkpoint and is not in the trained weights.)

3 · Training technique

Base	Qwythos-9B-v2, qwen3_5 Gated-DeltaNet, MLX 4-bit (group 64)
Adapter	LoRA r=32, α=64 (scale 2.0), dropout 0.05
Targets	12 modules — q/k/v/o_proj , gate/up/down_proj , and all five GDN leaves (in_proj_qkv , in_proj_z , in_proj_b , in_proj_a , out_proj)
Trainable	86.5 M / 9 B = 5.8%
Loss	completion-only , masked by tokenised prompt-prefix length
Objective	cut cross-entropy (CCE)
Optimiser	adamw_8bit , lr 1e-4, grad-accum 8, 1,941 steps
Seq length	4,096, gradient checkpointing on
Wall clock	43 h 28 m , peak 12.1 GB of 36 GB
Val loss	0.5786 → 0.5301 (step 400 → 1941)

Three engineering decisions did the heavy lifting:

Multi-turn unroll. A naive multi-turn row supervises the model on *tool output* as well as its own actions. I emit one row per assistant turn, with prior turns moved into the prompt. Median supervised span fell from 937 to 122 tokens, the model is trained only on what it should produce.

Pre-tokenisation. The trainer re-rendered the Jinja chat template across the whole dataset at startup, costing 30 minutes per run. Pre-tokenising to {input_ids, completion_mask} and asserting `full[:len(prompt)] == prompt` reduced that to 20 seconds, and removed a silent failure mode where a non-prefix render makes the loss mask fall back to length-based guessing.

Row capping. A single gold row containing a 2.2 KB base64 ccache blob reached 4,316 tokens. Per-row character caps with reserved space for the completion keep long lab trajectories trainable.

Diminishing returns were explicit. Validation loss improved by −0.025, −0.014, −0.008, −0.001 across the four checkpoints. The final third of the epoch bought almost nothing, which is why a second epoch on the same data was judged not worth 43 more hours, better data, not more passes.

4 · Results

4.1 InterCode-CTF, the primary, significant result

100 picoCTF-derived tasks, 15 turns, **temperature 0**, thinking disabled, 512-token cap. Trained = adapter fused, MLX 4-bit; base = same weights without the adapter at the same quantisation. Both models ran concurrently with separate containers.

model	solved
trained	10/100 · 10.0%
base	4/100 · 4.0%
paired over 100 tasks	
trained-only	6
base-only	0
both	4 · neither 90
McNemar (corrected)	$\chi^2 = 4.17$, $p < 0.05$
exact sign test	$p = 0.031$

Base's solves are a strict subset of trained's. Trained-only: task 4 (General, 9 turns), 5 (Crypto, 4), 11 (General, 10), 17 (General, 4), 40 (General, **2 turns**), 90 (Crypto, 4).

category	trained	base
General Skills	7/33	3/33
Cryptography	2/19	0/19
Forensics	1/15	1/15
Reverse Engineering	0/27	0/27
Binary Exploitation	0/4	0/4
Web Exploitation	0/2	0/2

Solving is bimodal. Solved tasks averaged 5.3 turns; every unsolved task exhausted all 15. The model recognises a task quickly or never, so raising the turn budget should not help, a prediction confirmed independently in §5.

4.2 CyBench under matched conditions

Re-run with identical quantisation, serving path, harness and day:

run	strict	content-match
trained	13/91 · 14.3%	16/91 · 17.6%
base (matched)	8/91 · 8.8%	11/91 · 12.1%

Task level: trained wins 5 ([Hard] Permuted , Flag Command , chunky , eval-me , network-tools), base wins 2 (PackedAway , Primary Knowledge), sign test $p = 0.4531$, **not significant**. Three independent base runs scored 11, 3 and 8 of 91; the fine-tune's 13 exceeds all three, which is suggestive but is not by itself a significance test. The instrument is too small and too noisy; 22 tasks with correlated subtasks cannot resolve an effect this size.

4.3 Efficiency, the largest and most consistent effect

Over the 12 tasks with comparable timings:

	total	mean/task
trained	2.90 h	870 s
base	4.71 h	1,413 s

Base is 1.62x slower and slower on 11 of 12 tasks (up to 2.88x). On [Very Easy] Flag Command base issued 58 commands and scored 0/4; trained scored 2/4. The fine-tune yields a terser agent reaching an equal-or-better score in ~60% of the wall time.

4.4 Knowledge regression, none detected

1,105-question web-pentest exam: trained **980/1105 (88.7%)**, base 990/1105 (89.6%). McNemar on 21-vs-31 discordant pairs: $\chi^2=1.56$, $p \approx 0.21$, not significant. 43 GPU-hours of offensive-security training did not measurably damage general knowledge. A probe also found that enabling the model's thinking path recovered 13 of 30 previously wrong answers (43%), indicating the non-thinking path used by these harnesses understates its ability.

4.5 Where it failed

Full-chain network penetration: on 21 in-vitro tasks the fine-tune and base each captured **1 flag**, on *different* tasks. Milestone re-scoring (127 ordered milestones) gave 47 vs 49 — still null. Both models shared three hard ceilings: running post-login commands on the attacker box rather than the target, never issuing UDP scans (which zeroes every SNMP task by construction), and never attempting a web exploit primitive.

Web exploitation: zero solves by any model on any web benchmark.

5 · Behavioural analysis: the model does not update on negative evidence

On a Django SSTI CTF solvable in **one request** (`{{ FLAG }}`), the model was given 20 turns, then 60. Budget was not the limiter.

With 60 turns it fixed a tooling failure unaided, it read `curl: (3) nested brace`, switched to `python3 http.client`, and cut empty responses from 15/20 to 1/60. It never fixed the conceptual error: **31 distinct payloads, all Jinja2/Flask idioms** against a Django engine that returned `TemplateSyntaxError` 20 times. It emitted `{% }`, Django's tag syntax, zero times. Closest approach was `{{ request.META.FLAG }}`: the right variable name, always wrapped in a namespace that does not exist.

Django renders an unknown variable as empty string, so ~14 wrong guesses returned an **identical null**. The model treated 14 indistinguishable nulls as 14 independent experiments, deepening one hypothesis rather than concluding the path was wrong and enumerating the context.

Hypothesis (not yet tested): the corpus records only successes, so it contains no examples of a failed probe driving a change of approach. Falsifier: count gold trajectories where a failed probe is followed by a change of approach; if that fraction is already substantial, this explanation is wrong.

6 · What I would change, using the same data

1. Add failure trajectories (§7). Highest expected value, no new corpus.
2. Rebalance toward execution. ~90% of rows are prose-derived; only 5.2% are real execution. The measured gains are in shell operation, precisely what the real execution traces teach.
3. **Train on stage6_web_exec** . 1,501 discovery-first web trajectories exist but postdate this checkpoint. Web capability is the clearest gap and this is the direct test of whether it is data-shape-limited or capability-limited. A prior analysis found the corpus was not missing payloads, 20% of rows carry one, but they target hard-coded write-up domains (201 public vs 83 private IPs), teaching "curl the named site" rather than *scan* → *enumerate* → *exploit*.
4. Remove the no-prose-leak gate. It taught `content: ""` , empty message bodies on 10 of 15 turns with thinking enabled, requiring a workaround on every benchmark and still breaking llama.cpp (§6).
5. Deduplicate validation. 5 of 150 held-out rows are byte-identical to training rows, inflating apparent validation quality.
6. Teach host awareness. Both models run post-login commands on the attacker box. Gold was complicit: the harness never recorded *which machine* a command ran on. Now fixed — observations are prefixed `[ ran on: <host> ]` , back-filled so 278 of 655 observations correctly show they ran on the target, but that fix postdates this checkpoint too.
7. Add UDP enumeration. Its absence zeroes every SNMP task by construction. Cheapest remaining win.

7 · Limitations

1. Single seed per model. Temperature 0 removes sampling variance but not environment nondeterminism. A 6–0 split is unlikely to be chance; multi-seed replication would settle it.
2. Narrow scope. No web, binary or reverse-engineering task was solved by either model.
3. Non-thinking path only. Harnesses cap output at 512 tokens with thinking off, testing a path the training barely touched (§5.4). The thinking path costs ~15 h per 1,105-item run.
4. **9B, 4-bit**. Both models at 4.50 bpw. The intended 27B did not fit.
5. Benchmark rot. CyBench loses ~6 of 40 tasks to dead Debian repositories; a web-CTF suite lost 3 of 12. Model-independent, but it inflates percentages and breaks comparability with published figures.
6. Contamination unmeasured. InterCode derives from picoCTF; the corpus includes public write-ups. Overlap was not quantified.

8 · Conclusion

One epoch of LoRA on 15,529 public offensive-security trajectories produced a statistically significant improvement in autonomous CTF solving: 10/100 vs 4/100 at $p < 0.05$, a strict superset of the base model's solves. It also cut wall-clock per task by **1.6x** with no detectable knowledge regression. Total marginal cost: 43 GPU-hours on a laptop.

The improvement is narrow and mechanistically legible: the corpus is overwhelmingly *operational* (command → observation → command), so the model got better at **driving a shell toward a goal** and no better at any vulnerability class it never practised in executable form. The two largest remaining defects are no learning from

failure and no executable web data in the trained mix. Both are data-composition problems, addressable with material already on disk.

Artifacts: `results_bank_20260923/` · `BEHAVIOUR_LEDGER.md` (*full experimental record*) ·
`results_ic_100.csv` · `INTERCODE_100_RESULT.txt` · `CYBENCH_TRAINED_GRADE.txt`